CTF Challenge Write-Up

Problem ID	Captured Flag	Steps (Summary)
P1 – Liars Dice	`fsuCTF{800t57r4p_8i11_y0u_4r3_4_1i4r_4nd_y0u_wi11_5p3nd_4n_37ernity_0n_7hi5_5hip}`	Brute-force microsecond seed for MT19937 PRNG using leaked dice, predict all dice every round, force AI to call liar
P2 – RSA Master	`fsuCTF{You_Can_Do_CRT_For_Any_E_If_You_Have_Enough_Ciphertexts}`	Collect 7 ciphertext/modulus pairs, apply Hastad's broadcast attack via CRT to reconstruct m^7, take integer 7th root
P3 – Jack The Ripper	`fsuCTF{John_Work_Well_With_Rockyou}`	Identify ZIPCrypto encryption, implement local password checker with correct header check byte, crack with rockyou.txt

P1 – Liars Dice

The server runs a game of Liar's Dice against the player. The AI 'cheats' by using full knowledge of the dice state to only make safe bids or call liar at the right time. The player's first 10 dice are printed each round, leaking PRNG output.

Vulnerability

The server seeds Python's random module with int(time.time()*1000000) the current time in microseconds. This seed space is small enough to brute-force. The first 10 dice values printed are the first 10 outputs of the MT19937 PRNG, which can be matched against candidate seeds:

```
seed = int(time.time() * 1000000)
random.seed(seed)
# First 10 dice are leaked to the player
```

Solution

1. Record the player's first 10 dice values from the server output
2. Brute-force nearby microsecond timestamps until a seed produces the same 10 values
3. With the seed recovered, predict both the player's and AI's dice for every round
4. Each round, bid the highest true statement covering all 20 dice
5. The AI is forced to call liar on a true bid and loses — repeat until the game is won

```
# Once seed is found:
random.seed(recovered_seed)
# Predict full dice state for all rounds
# Always bid a true statement the AI cannot safely call
```

Flag:
fsuCTF{800t57r4p_8i11_y0u_4r3_4_1i4r_4nd_y0u_wi11_5p3nd_4n_37ernity_0n_7hi5_5hip}

P2 – RSA Master

Each connection to the server prints a ciphertext and public key. The exponent is always e = 7 but the modulus changes each connection, while the same plaintext is encrypted every time. This is the classic setup for Hastad's broadcast attack.

Hastad's Broadcast Attack

Because the same message m is encrypted under 7 different moduli with exponent e = 7 and no padding, the Chinese Remainder Theorem (CRT) can reconstruct m^7 as an integer. Since m^7 is smaller than the product of all 7 moduli, taking the exact integer seventh root recovers m directly:

```
# Collect 7 pairs: (c_i, n_i) from 7 separate connections
# All encrypt the same plaintext m with e=7

# CRT reconstruction
M = n_1 * n_2 * ... * n_7
m_e = CRT([c_1,...,c_7], [n_1,...,n_7])  # = m^7 mod M

# Integer 7th root (exact, no modular reduction)
m = integer_nthroot(m_e, 7)
flag = m.to_bytes(...).decode()
```

Solution

1. Connect to the server 7 times and collect a ciphertext/modulus pair from each connection
2. Verify all 7 moduli are pairwise coprime (required for CRT)
3. Apply CRT to reconstruct m^7 over the integers
4. Compute the exact integer 7th root of the result to recover m
5. Decode m as bytes to get the flag string

Flag:
fsuCTF{You_Can_Do_CRT_For_Any_E_If_You_Have_Enough_Ciphertexts}

P3 – Jack The Ripper

The challenge provided a password protected ZIP archive containing a single file, flag.txt (36 bytes). The challenge name hinted at John the Ripper, a well known password cracking tool, and rockyou.txt as the wordlist.

Vulnerability

The archive used legacy ZIPCrypto encryption rather than modern AES. ZIPCrypto is a weak stream cipher that can be broken with dictionary attacks. The key detail was identifying the correct header check byte (0x06) used to validate password candidates:

```
ZIPCrypto header check byte: 0x06
  -> matches high byte of CRC/check value for this archive
  -> used to quickly reject wrong passwords before full decrypt
```

Solution

1. Parse the ZIP archive and identify ZIPCrypto (legacy) encryption on flag.txt
2. Implement a local ZIPCrypto password checker using the correct header check byte (0x06)
3. Run the checker over rockyou.txt to find the matching password
4. Decrypt flag.txt using the recovered password

```
# Password recovered from rockyou.txt:
password = 'Lejoshg'

# Decrypt and read flag.txt
```

Flag: **`fsuCTF{John_Work_Well_With_Rockyou}`**